

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ «ЛЭТИ» ИМ.В.И.УЛЬЯНОВА (ЛЕНИНА)»**

**ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ И ИНФОРМАТИКИ
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ**

**Зачётная работа №2
по дисциплине «Алгоритмы и структуры данных»
на тему «Множество как объект»**

Выполнили студенты группы 3312

Лебедев И.А.

Шарапов И.Д.

Принял старший преподаватель Колинко П.Г.

Санкт-Петербург
2024

Содержание

Цель работы	3
Описание задания	3
Контрольные тесты	3
Результаты эксперимента с четырьмя структурами данных на основе классов	5
Результат эксперимента с отслеживанием вызовов функций-членов	6
Выводы	6
Список использованных источников	8
Приложение	8

Цель работы

Сравнение процедурного и объектно-ориентированного подходов на примере задачи обработки множеств.

Описание задания

Множество десятичных цифр, которые одновременно принадлежат множествам A и B, а также цифры из множеств C и D.

$$E = A \cap B \cup C \cup D$$

Контрольные тесты

В первом примере (Рис. 1) пересечение множеств A и B даёт множество из двух элементов. После чего добавляются все элементы из C и D, так чтобы не было дубликатов.

```
Select a mode (Manual 'm' or automatic 'a'):
```

```
a
```

```
Enter length of set (1 to 10):
```

```
3
```

```
A: 156
```

```
B: 756
```

```
C: 046
```

```
D: 429
```

```
Array result:          [5, 6, 0, 4, 2, 9] in 8979 nanoseconds
```

```
List result:           [2, 9, 5, 6, 4, 0] in 4279 nanoseconds
```

```
Bool array result:     [0, 2, 4, 5, 6, 9] in 2199 nanoseconds
```

```
Machine word result:   [0, 2, 4, 5, 6, 9] in 819 nanoseconds
```

```
Process finished with exit code 0
```

Рисунок 1 – Пример с длиной множества 3

Во втором примере (Рис. 2) также можно наблюдать обработку дубликатов и корректное выполнение формулы.

```
Select a mode (Manual 'm' or automatic 'a'):
a
Enter length of set (1 to 10):
6
A: 974251
B: 184576
C: 408725
D: 027439

Array result:      [7, 4, 5, 1, 0, 8, 2, 3, 9] in 7398 nanoseconds
List result:       [3, 9, 7, 4, 5, 1, 2, 8, 0] in 7670 nanoseconds
Bool array result: [0, 1, 2, 3, 4, 5, 7, 8, 9] in 2584 nanoseconds
Machine word result: [0, 1, 2, 3, 4, 5, 7, 8, 9] in 841 nanoseconds

Process finished with exit code 0
```

Рисунок 2 – Пример с длиной множества 6

В третьем примере (Рис. 3) демонстрируется работа с множествами произвольной длины, введенными вручную.

```
Select a mode (Manual 'm' or automatic 'a'):
m
Enter the sets as a string consisting of numbers.
A:1234
B:4567
C:98
D:58

Array result:      [4, 9, 8, 5] in 8346 nanoseconds
List result:       [5, 4, 8, 9] in 2720 nanoseconds
Bool array result: [4, 5, 8, 9] in 1986 nanoseconds
Machine word result: [4, 5, 8, 9] in 651 nanoseconds

Process finished with exit code 0
```

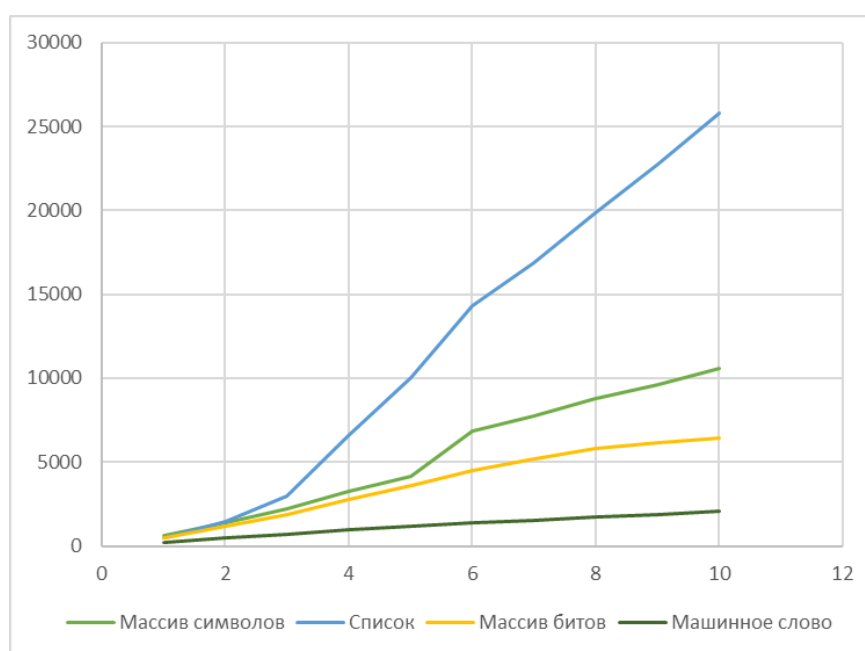
Рисунок 3 – Пример ручного ввода множеств

Результаты эксперимента с четырьмя структурами данных на основе классов

Таблица 1. Результаты измерения времени обработки множеств.

Мощность множеств	Количество <i>наносекунд</i> при обработке множеств при различных способах представления			
	<i>Массив символов</i>	<i>Список</i>	<i>Массив битов</i>	<i>Машинное слово</i>
1	606	471	495	244
2	1393	1479	1153	467
3	2192	2994	1896	688
4	3278	6580	2766	938
5	4137	10025	3601	1153
6	6833	14295	4474	1375
7	7775	16884	5217	1556
8	8761	19868	5780	1733
9	9645	22781	6165	1894
10	10553	25820	6428	2051

Диаграмма 1. Оценка измерения времени обработки множеств.



Результат эксперимента с отслеживанием вызовов функций-членов

```
Select a mode (Manual 'm' or automatic 'a'):  
a  
Enter length of set (1 to 10):  
5  
A: 71829  
B: 08657  
C: 30651  
D: 12509  
Set string constructor called for Set A  
Set string constructor called for Set B  
Set string constructor called for Set C  
Set string constructor called for Set D  
Set operator & called for Set A and Set B  
Set default constructor called for Set E  
Set operator | called for Set E and Set C  
Set copy constructor called for Set F from Set E  
Set operator | called for Set F and Set D  
Set copy constructor called for Set G from Set F  
Set Show called for Set G = [783065129]  
Set destructor called for Set G  
Set destructor called for Set F  
Set destructor called for Set E  
Set destructor called for Set D  
Set destructor called for Set C  
Set destructor called for Set B  
Set destructor called for Set A  
  
Process finished with exit code 0
```

Выводы

В работе была выполнена задача обработки множеств с использованием объектно-ориентированного подхода и были рассмотрены ключевые преимущества этого подхода по сравнению с традиционным процедурным методом. Были реализованы несколько классов, каждый из которых представил множество в различных формах, таких как машинное слово, массив битов, массив символов и связный список.

Преимущества объектно-ориентированного подхода:

1. Инкапсуляция.

Классы позволяют скрыть внутреннее представление данных и предоставляют чётко определённый интерфейс для работы с множествами. Благодаря этому можно легко изменять способ хранения множества

(например, с массива на булевый массив или машинное слово), не влияя на остальной код.

2. Перегрузка операторов.

Благодаря перегрузке операторов (например, `|`, `&`, `~`), стало возможным выполнять операции объединения, пересечения и дополнения так же просто, как и для встроенных типов данных. Это повышает читаемость и удобство использования кода.

3. Переиспользование кода.

Объектно-ориентрованное программирование позволяет повторно использовать классы для различных задач и применять единый интерфейс работы с множествами, независимо от способа их хранения. Это облегчает расширение функциональности программы и улучшает её поддержку.

4. Гибкость.

Объектно-ориентированный подход позволяет легко добавлять новые методы и расширять функциональность классов без внесения изменений в существующий код. Например, добавление новых методов для обработки множеств не требует переписывания программы, как это было бы при использовании процедурного подхода.

Сравнение с процедурным подходом:

При процедурном подходе перегрузка операторов недоступна, что делает код более громоздким и сложным для восприятия при выполнении операций над множествами. Объектно-ориентированный подход, напротив, обеспечивает удобную работу с множествами через перегрузку операторов, что делает взаимодействие с классами более интуитивным.

При использовании процедурного подхода управление множествами, как правило, требует написания множества функций, каждая из которых должна быть совместима с выбранным способом представления данных. При изменении структуры данных необходимо менять и функции, что усложняет код и делает его менее гибким.

Объектно-ориентированный подход, напротив, позволяет создавать абстракции, которые объединяют данные и функции в единый интерфейс (класс). Это минимизирует необходимость в изменении кода при добавлении новых возможностей или при изменении способа представления множества.

Однако стоит отметить, что объектно-ориентированный подход может быть несколько медленнее по времени выполнения по сравнению с процедурным подходом. Это связано с дополнительными накладными расходами, возникающими при работе с объектами и вызовами методов. Тем не менее, преимущества ООП, такие как модульность, переиспользование кода и удобство разработки, часто перевешивают небольшое снижение производительности в большинстве случаев.

Список использованных источников

1. Колинко П. Г. Пользовательские структуры данных: Методические указания по дисциплине «Алгоритмы и структуры данных, часть 1». — СПб.: СПбГЭТУ «ЛЭТИ», 2024. — 64 с. (вып.2408).
2. Алгоритмы и структуры данных. Лекции на сайте <https://vec.etu.ru/moodle/course/view.php?id=13286>.

Приложение

- Архив с основным кодом main.zip (ветка [task2](#))
- Архив с кодом для сравнения времени main-test.zip (ветка [task2_tests](#))
- Архив с кодом для отслеживания вызова функций-членов main-image.zip (ветка [task2_image](#))